

HiggsCP Analysis

DOCUMENTATION AND GUIDES

APRIL 2024

Table of Contents

1. Configuring The Environment
2. ML Flow
3. Model Architecture
4. Command Line Options
5. Examples

CONFIGURING THE ENVIRONMENT

This section shows how to create a virtual environment on **Linux** (though also tested on **MacOS**) and then install all the necessary dependencies. You are expected to have the **Python 3.11.3** interpreter ([Python Software Foundation official page](#)) and **Git** ([Project Git official page](#)) installed on your computer.

If you are going to configure your working environment on Ares clusters (PLGrid, Cyfronet infrastructure), you need to allocate resources and then load the appropriate Python module. The following command will allocate one CPU for 1 hour and then load the interpreter we need:

```
$ srun --time 00-01:00:00 --pty /bin/bash
$ module load python/3.11.3-gcccore-12.3.0
```

Let's now clone the repository, create a new virtual environment in it, activate the environment, update pip (package-management system) and install all the libraries stated in `requirements.txt` (available in the repository):

```
$ git clone https://github.com/klasocha/HiggsCP.git
$ cd HiggsCP
$ git checkout tyerniyazov-rhorho_refactored
$ python3.11 -m venv .venv
$ source .venv/bin/activate
(.venv) $ python -m pip install --no-cache-dir --upgrade pip
(.venv) $ pip install --no-cache-dir -r requirements.txt
```

The process of installing the libraries may take a while to finish. When it completes, your working environment is ready to run the code from the cloned repository. For Ares users, the first two steps in the violet box are regular. For all users, the step involving activating the virtual environment is regular. Don't forget to deactivate the environment if you finish experimenting with the project:

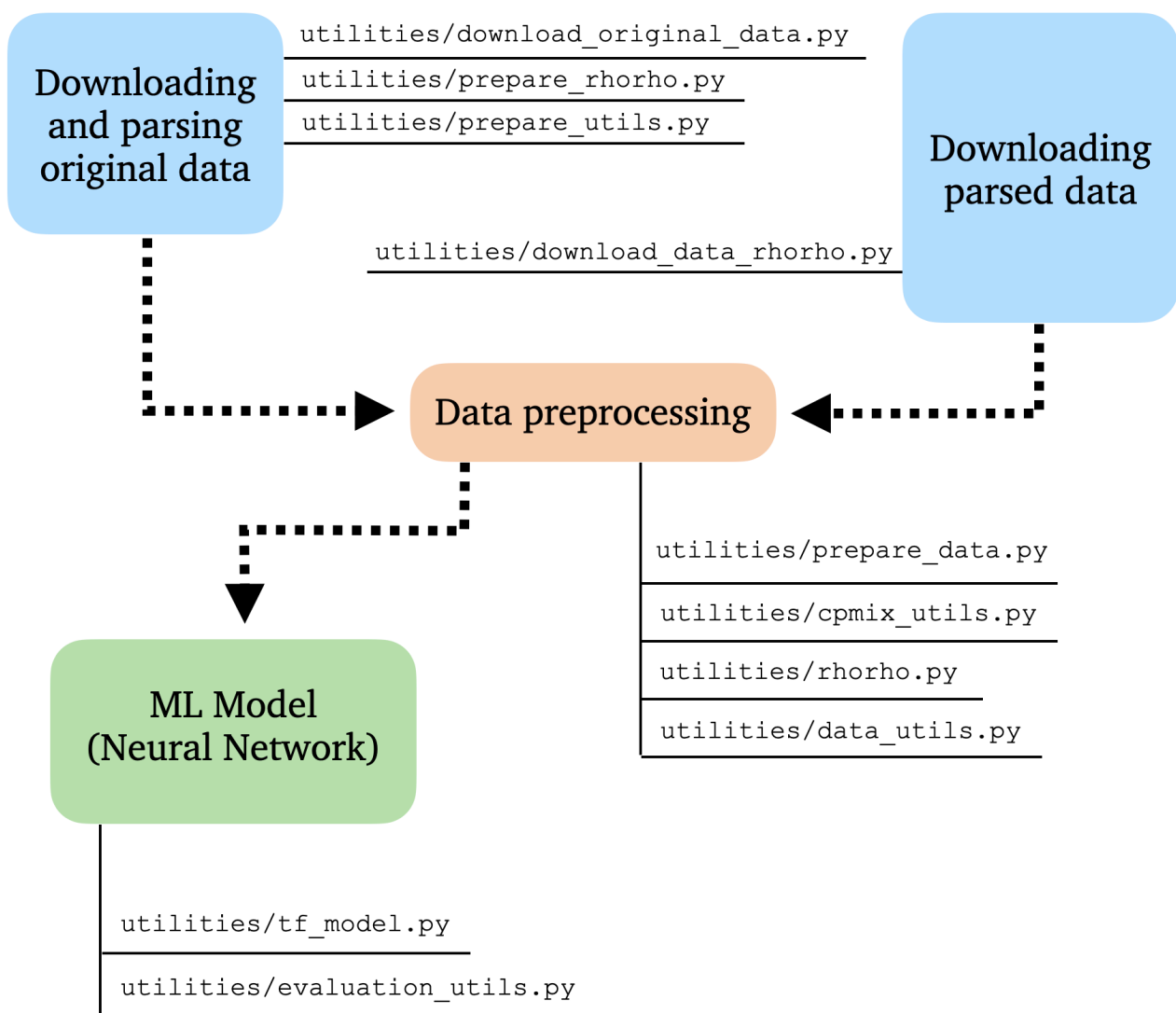
```
# Activating
$ source .venv/bin/activate

# Deactivating
(.venv) $ deactivate
```

ML FLOW

The whole project contains a number of packages responsible for implementing different parts of the machine learning flow. Each package contains several modules which are used by the main program `main.py` invoking particular actions, such as downloading data, training the model or even preparing some plots.

The below diagram depicts the main parts of the ML pipeline and the names of some of the modules involved:

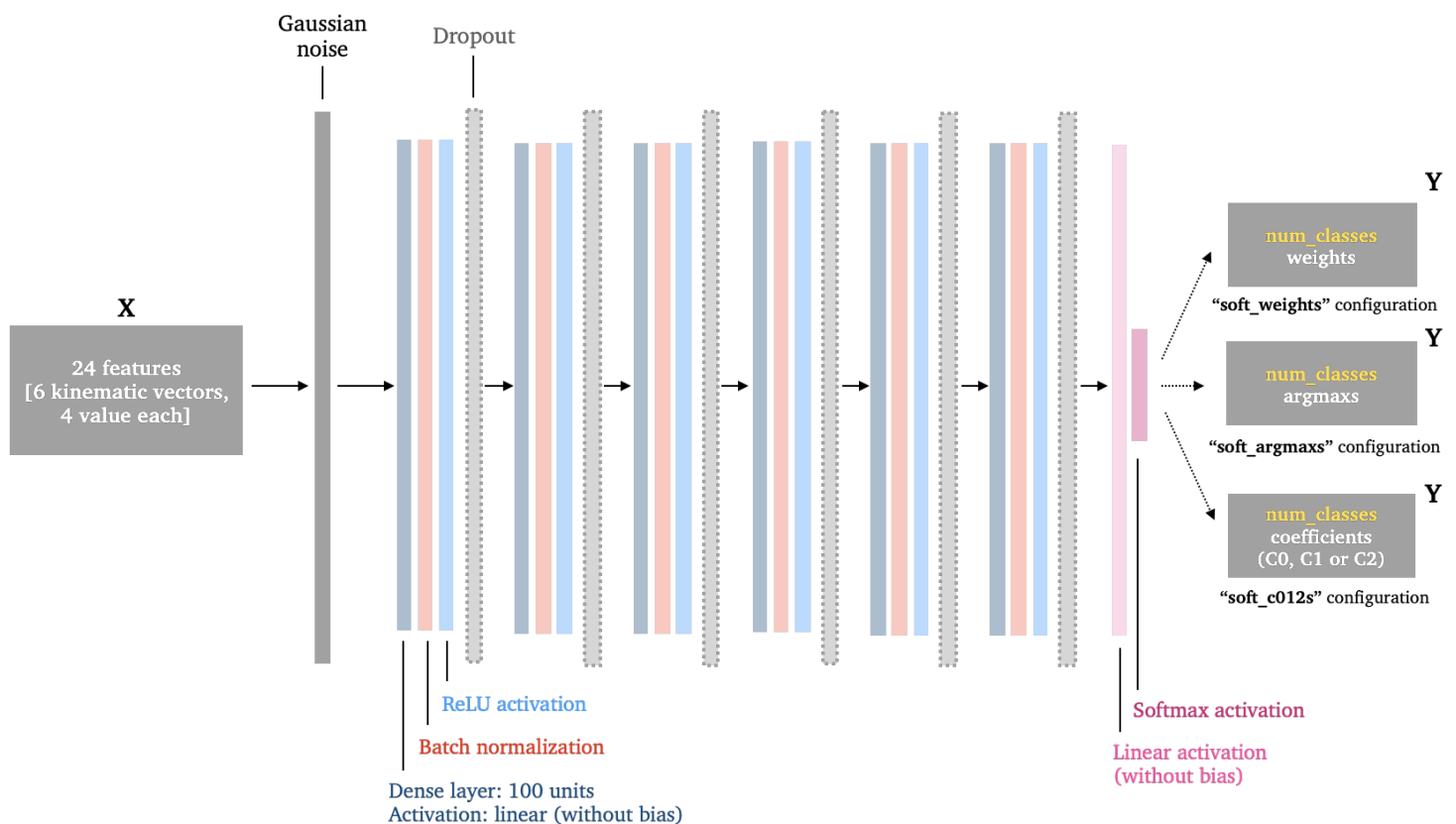


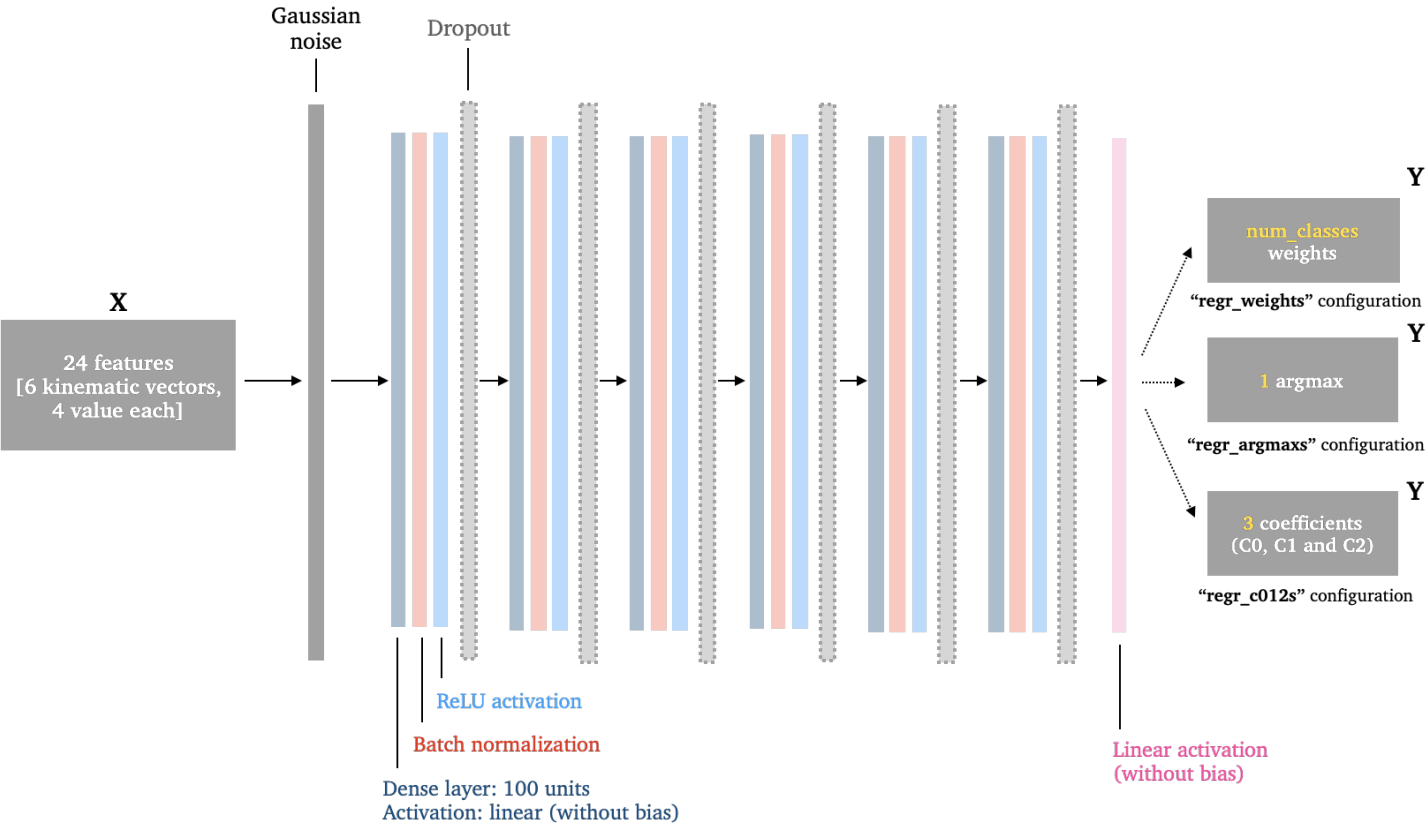
MODEL ARCHITECTURE

The ML model is represented by an artificial neural network which takes 6 different configurations. Depending on the type of configuration the architecture changes. Additionally, it is possible to specify some of the hyperparameters affecting the model structure and the training process (optimizer, loss function, learning rate etc.)

The two diagrams below show the whole architecture for those configurations:

- „soft_weights”
- „soft_argmaxs”
- „soft_c012s” (**only one type of coefficients is trained at once**)
- „regr_weights”
- „regr_argmaxs”
- „regr_c012s” (**all three types of coefficients are trained simultaneously**)





COMMAND LINE OPTIONS

Whilst there are a number of options available, we will consider the most important ones in this section. To see the full list of the command line arguments, you can use the `--help` option while calling `main.py`:

```
$ python main.py --help
```

--input [data path] - the path to the directory containing the data sets. If you download the data sets for the first time, this directory will be created automatically.

--num_classes [number of classes] - how many classes are going to be used for discretisation. This also impacts the model architecture as the output shape may change (see section „Model Architecture”).

--hits_c012s [hits_c0s / hits_c1s / hits_c2s] - defines which coefficients (C0, C1 or C2) to choose as labels during the training process.

--features [input features choice] - choosing the set of features for training. You may choose one of the following values: VariantAll, Variant-1.0, Variant-1.1, Variant-2.0, Variant-2.1, Variant-2.2, Variant-3.0, Variant-3.1, Variant-4.0, Variant-4.1.

--training_method [configuration] - choosing the model type. It defines the architecture of the model (see section „Model Architecture”). You may choose one of the following types: `soft_weights`, `soft_c012s`, `soft_argmaxs`, `regr_c012s`, `regr_weights`, `regr_argmaxs`.

The following options allow you to control some of the hyperparameters applied during the training process:

--dropout [rate] - dropout probability (applied during the training process).

--optimizer - the type of optimizer. You may choose one of the following (the default value for this option is `AdamOptimizer`):

- `GradientDescentOptimizer`,
- `AdadelatOptimizer`,
- `AdagradOptimizer`,
- `ProximalAdagradOptimizer`,
- `AdamOptimizer`,
- `FtrlOptimizer`,
- `ProximalGradientDescentOptimizer`,
- `RMSPropOptimizer`

--epochs [epochs] - the number of epochs used during the training process.

The next option is important for specifying the path to the model weights, training history, configuration file, as well as the directory containing predictions:

--model_location [directory name] - specifying the name of the directory in `results/` containing the model state (weights, metadata).

The following group of arguments impacts the way in which different plots are prepared (more details are available in the source code of the modules responsible for drawing the plots. You can find them all located in `plots/`):

--output [path] - output path for plots

--format [format] - the format of the output plots ("png"/"pdf"/"eps")

--show - use it to display the plots before saving them

--option - specifying the plotting module to call. You may choose one of the following:

- PHISTAR-DISTRIBUTION
- C012S-WEIGHT
- C012S-DISTRIBUTION
- WEIGHTS-FOR-EVENT-VIA-C012
- UNWEIGHTED-EVENTS-WEIGHTS

--hypothesis [hypothesis class index] - the alphaCP class (e.g. 02)

The last option we should take a look at before running the main program is about choosing the action controlling the ML flow behaviour (see section „*Examples*“):

--action [action] - choosing the action. Available values:

- download_and_prepare_original
- download_and_preprocess
- train
- continue_training
- predict
- plot
- test

EXAMPLES

Take a look at some of the examples to see how `--action` controls what modules are called by `main.py`:

Downloading and preprocessing the original (raw) data sets

```
$ python main.py --action "download_and_prepare_original" --input  
„data_original"
```

Downloading those data sets that are available on the authors' server

and that already contain the set of parsed kinematic vectors and weights

```
$ python main.py --action "download_and_preprocess" --input "data" --  
features "Variant-All" --num_classes "11"
```

Training the model on 5 epochs for the „soft_weights” configuration:

```
$ python main.py --action "train" --input "data" --num_classes "11" --  
epochs "5" --training_method "soft_weights" --model_location „model_1"
```

Using the pre-trained model weights to make predictions on the validation and training sets:

```
$ python main.py --action "predict" --input "data" --num_classes "11" --  
model_location „model_1"
```